

Rapport exercice IV

I Présentation de l'exercice

Le but de l'exercice était d'implémenter une solution au problème du sac à dos. Pour cela, j'ai implémenté un algorithme détaillé [ici](#) et [ici](#).

De plus, il nous était demandé de paralléliser le programme à l'aide d'OpenMp.

L'implémentation est trouvable sur [GitHub](#) dans le fichier Ex4.

II Calcul du temps

Afin d'éviter les incohérences dues à la mise en cache, j'ai créé un script python [Chapitre V.3] qui exécute le programme avec des arguments d'exécution différents à chaque fois.

Et de plus, pour plus de stabilité, je calcule la moyenne des temps sur 200 exécutions.

III Résultats

Selon les fichiers de test fournis, les résultats ne sont pas les même.

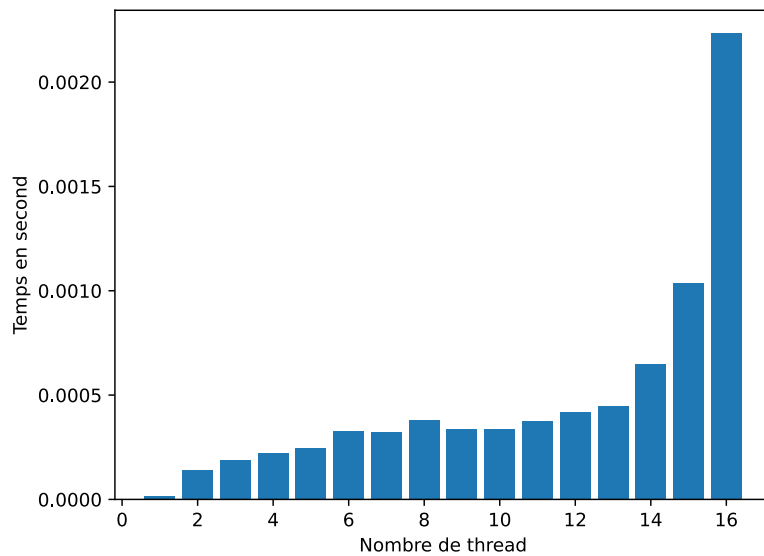
III.1 pb1

Pour le premier fichier de test, on peut observer ces temps

Temps pris par l'implémentation pour pb1.pb.

Nombre de thread	Temps pris
1	0.015409ms
2	0.14212ms
3	0.189889ms
4	0.221029ms
5	0.24569ms
6	0.32834ms
7	0.32435ms
8	0.38077ms

Nombre de thread	Temps pris
9	0.33865ms
10	0.33844ms
11	0.37649ms
12	0.417239ms
13	0.44612ms
14	0.646729ms
15	1.03853ms
16	2.232589ms



Nous remarquons que le temps augmente en fonction du nombre de threads, pb1 étant très petit (seulement 5 objets), le temps d'attente des threads écrase vite celui de l'exécution mono-thread.

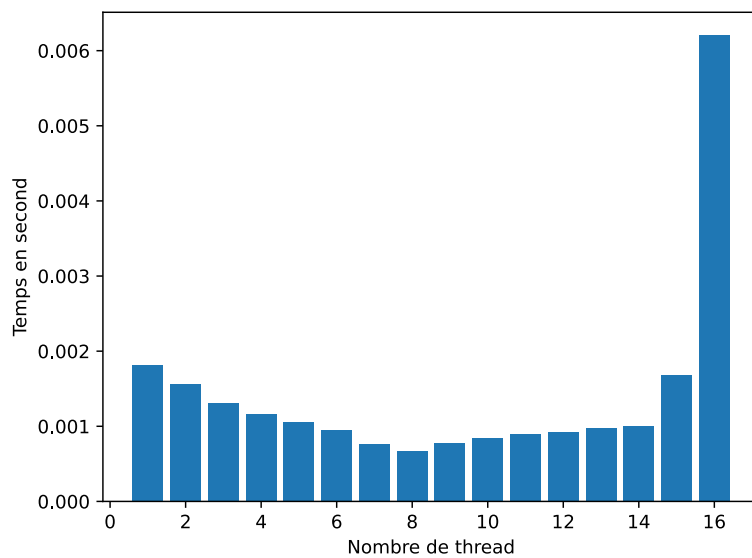
III.2 pb2

Ensuite, pour le second test, nous avons :

Temps pris par l'implémentation pour pb2.pb.

Nombre de thread	Temps pris
1	1.811889ms
2	1.55808ms
3	1.30556ms
4	1.168409ms
5	1.050529ms
6	0.95588ms
7	0.764639ms
8	0.674949ms

Nombre de thread	Temps pris
9	0.77126ms
10	0.844039ms
11	0.894239ms
12	0.919879ms
13	0.973499ms
14	1.00733ms
15	1.687449ms
16	6.20095ms



Nous pouvons observer une amélioration du temps jusqu'à environ 8 threads, puis une augmentation du temps. Ce dernier augmente de façon significative pour 16 thread.

L'explosion de fin est probablement due à un problème de cohérence de cache, en effet, les threads doivent tous rentrer en concurrence pour la lecture/écriture dans les blocs de cache, ce qui explique cette nette augmentation.

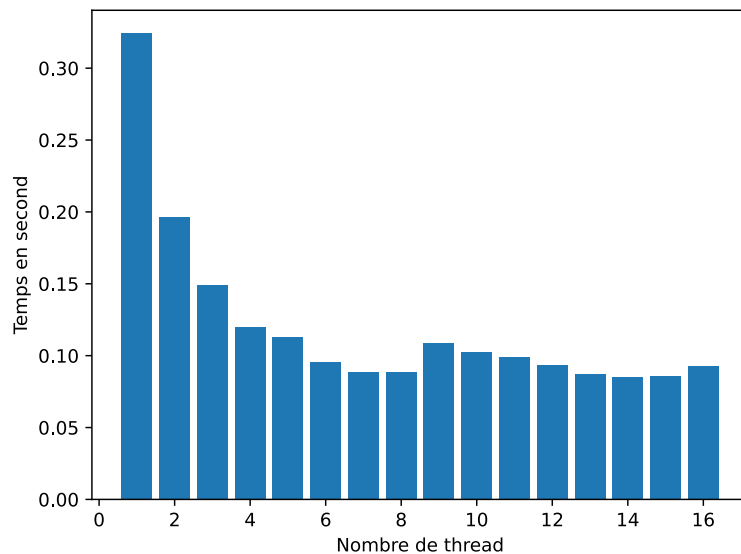
III.3 pb6

Et enfin pour pb6 :

Temps pris par l'implémentation pour pb6.pb.

Nombre de thread	Temps pris
1	324.06527ms
2	196.7247ms
3	148.88366ms
4	120.033649ms
5	112.58232ms
6	95.300769ms
7	88.374639ms
8	88.59447ms

Nombre de thread	Temps pris
9	108.79032ms
10	102.215169ms
11	98.84457ms
12	93.7129ms
13	87.39626ms
14	84.801589ms
15	85.82016ms
16	92.63982ms



Le nombre de threads améliore le temps d'exécution, et semble ne plus trop avoir d'impact entre 8 et 16 threads. Nous ne sommes pas confrontés à un problème de cache similaire à celui de pb2, car ici, les données étant très larges, les threads n'entrent pas en concurrence pour la lecture/écriture dans les blocs de cache.

IV Conclusion

Nous pouvons conclure notre analyse avec la loi d'Amdahl : L'accélération S_n peut être calculée à partir de : $S_n := \frac{T_1}{T_n} = \frac{1}{s + \frac{1-s}{N}}$

Avec :

T_1 := "Temps d'exécution de l'algorithme en séquentiel."

T_n := "Temps d'exécution de l'algorithme pour n threads en parallèle" et

s := "Temps d'exécution de la partie non parallélisable du programme."

De plus $S_n \xrightarrow{n \rightarrow +\infty} \frac{1}{s}$

Pour pb2 et pb6 la loi d'Amdahl paraît bien être respectée, avec une accélération qui semble se stabiliser vers la fin (mis à part les problèmes de caches, non pris en compte par la loi d'Amdahl).

V Annexe

V.1 Machine utilisé

Tous les temps ont été enregistrés sur un ordinateur portable avec un processeur x86-64 de huit cœurs physiques dont deux virtuels par cœur, cadencé à 3,5 GHz (sur secteur).

V.2 Données récolter

Toutes les mesures effectuées sont stockées dans un document *json* récupérer grâce à la sortie standard du programme, et à un script Python, trouvable sur le [GitHub](#).

V.3 Code

L'entièreté du code se trouve sur le [GitHub](#) dans le fichier Ex4.

V.4 Notice sur l'utilisation de l'IA générative

L'IA générative n'a pas été utilisée pour cet exercice.